

Estándares de Codificación

control-asistencia-backend

Sistema Inteligente de Control de Asistencia

Documento elaborado a partir de una auditoría de código

Proyecto: control-asistencia-backend

Repositorio: github.com/jacksonmontoya243-design/control-asistencia-backend

Rama: main

Versión del proyecto: 0.0.1-SNAPSHOT

Fecha de auditoría: 11 de agosto de 2026

Autor: Jackson Montoya Mercado

Índice

1. Introducción
2. Resumen de la auditoría
3. Arquitectura y estructura de paquetes
4. Convenciones de nomenclatura
5. Formato y estilo de código
6. Comentarios y documentación
7. Estándares por capa
8. Seguridad (reglas transversales)
9. Pruebas (tests)
10. Configuración y perfiles
11. Git y control de versiones
12. DevOps: Docker y CI/CD
13. Deuda técnica y acciones recomendadas

1. Introducción

Este documento define los estándares de codificación del backend **control-asistencia-backend**, un sistema full-stack para el registro y seguimiento de entradas y salidas del personal mediante códigos QR individuales. Los estándares han sido derivados del análisis en modo auditoría del código fuente actual, de modo que reflejan de manera exacta las convenciones ya aplicadas en el proyecto y sirven como guía oficial para futuras contribuciones.

Stack tecnológico: Java 17, Spring Boot 3.5.14 (Web, Data JPA, Security, Validation), Maven, PostgreSQL 16, ZXing y HTML5-QR Code, JWT 0.12.6, Lombok y Logback. Como frontend se integra un módulo React 19 + Vite 8 (carpeta *frontend-asistencia*).

2. Resumen de la auditoría

El código fue revisado capa por capa (controller, dto, entity, exception, repository, security, service), junto con la configuración, los tests, la infraestructura Docker y los flujos CI/CD. El proyecto presenta una arquitectura limpia y consistente. Los hallazgos principales son:

Resultado	Hallazgo	Acción requerida
OK - Consistente	Uso uniforme de records inmutables para DTOs con validación Bean Validation.	Conservar como estándar.
OK - Consistente	Inyección por constructor y dependencias finales en la capa de servicios.	Conservar como estándar.
OK - Consistente	Manejo global de errores con formato unificado { status , mensaje , errores }.	Conservar como estándar.
OK - Seguro	No se detectaron credenciales ni secretos commiteados (.env ignorado, JWT_SECRET por entorno).	Mantener política de secretos.
OK - Seguro	Autorización por roles con @PreAuthorize ; registro sin escalada de privilegios.	Conservar.
DEUDA	<i>EmpleadoController</i> usa @Autowired sobre campos; <i>UserController</i> usa constructor.	Unificar a inyección por constructor.
DEUDA	Ruta de QR static/qr/ fija en <i>QrGeneratorService</i> .	Mover a app.qr.path por entorno.
INCONSIST.	<i>Empleado</i> usa getters/setters manuales; <i>Asistencia</i> y <i>Usuario</i> usan Lombok.	Estandarizar con Lombok.
PENDIENTE	Faltan tests de integración (H2) y de <i>AuthController</i> , <i>UserController</i> y <i>AsistenciaController</i> .	Ampliar cobertura.
RIESGO	Rate limiting en memoria (una instancia); multi-instancia requeriría Redis.	Evaluar store distribuido.

3. Arquitectura y estructura de paquetes

El backend sigue una arquitectura en capas. Todo el código vive bajo el paquete base **com.example.controlasistenciabackend** y se organiza por responsabilidad técnica:

```
com.example.controlasistenciabackend
|-- controller/    # Capa REST: expone los endpoints /api
|-- dto/          # Records de request/response con validación
|-- entity/       # Entidades JPA que mapean tablas
|-- exception/    # Manejador global de excepciones
|-- repository/   # Interfaces Spring Data JPA
|-- security/     # JWT, filtros, rate limiting, UserDetails
'-- service/      # Lógica de negocio
```

Reglas de la arquitectura: los controladores solo orquestan la petición y delegan en servicios; los servicios contienen la lógica de negocio y usan DTOs en los límites; los repositorios encapsulan el acceso a datos; las entidades nunca se reciben directo como cuerpo de petición (se usa un DTO, ver *EmpleadoRequest*). La responsabilidad se organiza por capa, no por feature, manteniendo un flujo unidireccional controller → service → repository.

4. Convenciones de nomenclatura

Elemento	Convención	Ejemplos en el proyecto
Clases / interfaces / enums	PascalCase	EmpleadoController, AsistenciaService, Role
Métodos	camelCase, verbo en español	listar, guardar, actualizar, eliminar, obtener
Atributos y variables	camelCase	empleadoId, fechaHora
Parámetros	camelCase	empleadoId, desde, hasta, termino
Constantes / Logger	UPPER_SNAKE_CASE	private static final Logger log
Constantes de enum	UPPER_SNAKE_CASE	ADMIN, SUPERVISOR, EMPLEADO, ENTRADA, SALIDA
Paquetes	minúsculas, basado en dominio	com.example.controlasistenciabackend
DTOs	Nombre + Request/Response	EmpleadoRequest, AuthResponse, LoginRequest
Controladores	Sustantivo + Controller	EmpleadoController, AuthController
Servicios	Sustantivo + Service	AsistenciaService, QrGeneratorService
Métodos derivados JPA	findBy..., countBy..., ... OrderByFechaHoraDesc	findByEmpleadoIdOrderByFechaHoraDesc

4.1 Nombres de métodos en español

Los nombres de métodos se escriben en español con verbo en infinitivo cuando son acciones sobre el dominio: *registrarAsistencia*, *cambiarRol*, *buscarEmpleados*, *generarToken*. Los nombres de tests combinan *método_condición_resultadoEsperado*, también en español (ver sección de pruebas).

4.2 Convención de endpoints

Los endpoints usan el prefijo **/api**, sustantivo en plural y subrecursos para acciones específicas. Los filtros se pasan como query params y los identificadores como path variables:

```
GET    /api/empleados?termino=ana&page=0&size=10
POST   /api/empleados
PUT    /api/empleados/{id}
DELETE /api/empleados/{id}
GET    /api/asistencias/reporte?empleadoId=1&desde=...&hasta=...
```

4.3 Configuración

Las claves de configuración propias usan el prefijo **app.** en minúsculas kebab-case, y las variables de entorno se escriben en UPPER_SNAKE_CASE:

```
app.jwt.secret=${JWT_SECRET:...}
app.jwt.expiration=${JWT_EXPIRATION:86400000}
app.qr.base-url=${APP_QR_BASE_URL:http://localhost:8080}
app.ratelimit.max-requests=100
app.admin.username=${ADMIN_USERNAME:admin}
app.security.requires-secure=true
```

5. Formato y estilo de código

- Indentación de 4 espacios; llaves de apertura al final de la línea (estilo K&R).
- Máximo $\approx$ 120 caracteres por línea; líneas de métodos largos separadas en parámetros por línea.
- Imports agrupados: primero Java/Jakarta, luego dependencias de terceros, luego clases del proyecto.
- Una responsabilidad por clase y métodos cortos con nombres descriptivos.
- Uso de var para tipos obvios (p. ej., autoridades) y tipos explícitos en lo demás.

6. Comentarios y documentación

- Todo el código, los mensajes de error y los logs se escriben en **español**.
- Javadoc obligatorio en clases y métodos públicos: describe qué hace, con @param / @return cuando aplican.
- Comentarios de pasos numerados (1. Búsqueda, 2. Validación, 3. Persistencia) en lógica compleja o tipo tutorial.
- Comentarios que aclaran el *por qué* (p. ej., prefijo ROLE_ para hasRole(), rol EMPLEADO anti-escalada).
- Los logs usan SLF4J con placeholders {} y nivel coherente: info (crear/actualizar/eliminar/login), debug (detalle), warn (recurso inexistente), error (fallos).
- No se documentan comportamientos obvios; el código debe leerse como documentación.

7. Estándares por capa

7.1 Controladores (controller)

- @RestController + @RequestMapping("/api/<recurso>") al nivel de clase.
- @PreAuthorize("hasRole('ADMIN')") o @PreAuthorize("hasAnyRole('ADMIN','SUPERVISOR')") a nivel de método o de clase (ver UserController).

- Los cuerpos de petición se validan con `@Valid @RequestBody`: nunca recibir la entidad directamente, usar un DTO.
- Parámetros opcionales con `@RequestParam(required = false)` y fechas con `@DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME)`.
- Responder con `ResponseEntity` cuando el estado HTTP debe ser explícito: 201 Created, 204 No Content, 404 Not Found.
- Inyección de dependencias **por constructor** (objetivo del estándar; los `@Autowired` de campo se migrarán).
- Métodos delgados: obtienen el dato del DTO, delegan en el servicio y devuelven el resultado.

7.2 Servicios (service)

- `@Service` y dependencias finales inyectadas por constructor.
- Contienen la lógica de negocio y las validaciones de dominio; no exponen la entidad hacia afuera.
- Métodos de mapeo privados `toResponse(...)` convierten entidad → DTO de respuesta.
- Excepciones de dominio: `IllegalArgumentException` para conflictos, `NoSuchElementException` para "no encontrado", `IllegalStateException` para reglas de negocio.
- Uso de streams y lambdas para transformaciones de colecciones.
- Logger privado por clase: `private static final Logger log`.

7.3 Repositorios (repository)

- Interfaces que extienden `JpaRepository<Entidad, Long>` anotadas con `@Repository`.
- Consultas derivadas por convención de nombre; comentarios describiendo la consulta.
- Nombres que reflejan el filtro y el orden: `findByEmpleadoIdAndFechaHoraBetweenOrderByFechaHoraDesc`.
- Para agregados numéricos se usan métodos `countBy...`

7.4 Entidades (entity)

- `@Entity + @Table(name = "<plural>")`: el nombre de tabla siempre explícito.
- `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)` para la clave primaria Long y sin setter para el id (autoincremental).
- `@Column` con restricciones explícitas (`nullable = false, unique = true`).
- `@Enumerated(EnumType.STRING)` para enums persistentes (`Role, TipoAsistencia`).
- Estándar de boilerplate: Lombok `@Data + @Builder + @NoArgsConstructor + @AllArgsConstructor` (ver *Asistencia* y *Usuario*). *Empleado* aún usa getters/setters manuales y será migrado.

7.5 DTOs (dto)

- Se definen como **records** inmutables — datos de solo lectura.
- Validación Bean Validation sobre componentes: `@NotNull / @NotBlank` con mensajes en español por defecto.
- Sufijos Request/Response según el uso (`AsistenciaRequest, AuthResponse, UserResponse`).

7.6 Seguridad (security)

- JWT HS256 con JJWT; firma y caducidad configurables (`app.jwt.secret, app.jwt.expiration`).
- Sesiones stateless (`SessionCreationPolicy.STATELESS`) y CSRF deshabilitado (API con JWT).
- Contraseñas siempre con `BCryptPasswordEncoder`; no se guardan en claro.

- Filtros como componentes Spring (`JwtAuthenticationFilter`, `RateLimitFilter`) registrados antes que `UsernamePasswordAuthenticationFilter`.
- Autoridad con prefijo `ROLE_` para que funcione `hasRole()`: `new SimpleGrantedAuthority("ROLE_" + role)`.
- Rate limiting por IP con ventana deslizante; confiar en `X-Forwarded-For` solo con IPs válidas.
- Cabeceras de seguridad: `HSTS` (prod), `X-Frame-Options DENY`, `nosniff`; `HTTPS` forzado en prod.

Reglas de autorización: públicos login y `/qr/**`; gestión de usuarios solo ADMIN; creación/actualización/eliminación de empleados solo ADMIN; lectura de empleados ADMIN/SUPERVISOR; asistencias para cualquier usuario autenticado.

7.7 Manejo de errores (exception)

- `GlobalExceptionHandler` con `@RestControllerAdvice` centraliza todas las respuestas de error.
- Formato unificado de salida: `{ "status": <código>, "mensaje": <texto>, "errores": <detalle>? }`.
- Sentidos estándar: 400 validación/argumento, 401 credenciales/no autenticado, 403 acceso denegado, 404 no encontrado, 500 error interno.
- Nunca exponer stack traces al cliente; solo mensajes controlados en español.
- El manejador genérico de `Exception` es la red de seguridad final.

8. Seguridad (reglas transversales)

- Ninguna credencial, secret, token o archivo de entorno se commitea (`.env`, `*.key`, `*.pem` ignorados).
- Los artefactos generados no se versionan (`target/`, `node_modules/`, `dist/`, `static/qr/`).
- Anti-escalada de privilegios: el registro de usuario siempre asigna rol EMPLEADO; los roles superiores solo por ADMIN.
- Anti auto-eliminación: no se permite borrar el propio usuario administrador.
- Secreto JWT obligatorio en producción (sin valor por defecto en el perfil prod).

9. Pruebas (tests)

- JUnit 5 + Mockito (`MockitoExtension`) para servicios con `@Mock` y `@InjectMocks`.
- `@WebMvcTest` + `MockMvc` + `@MockitoBean` para controladores y `@WithMockUser(roles = "ADMIN")` para autorización.
- Nombres de tests en español: *método_condición_resultadoEsperado* (p. ej. `registrarUsuario_conUsernameExistente_lanzaIllegalArgumentException`).
- Cubrir casos felices y negativos: validación (400), no encontrado (404), credenciales inválidas (401), autorización.
- Comando oficial de ejecución: `mvn -B test` (también usado por CI).

10. Configuración y perfiles

- Perfiles explícitos: `application-dev.properties` (desarrollo) y `application-prod.properties` (producción), activados por `SPRING_PROFILE`.
- Dev: SQL visible, logs DEBUG, rate limit y HTTPS desactivados. Prod: SQL oculto, logs INFO, rate limit 100 req/min y HTTPS forzado.
- En prod las claves sensibles no tienen valor por defecto y deben inyectarse por variables de entorno.

- Nombre de la app y prefijo de propiedades consistentes (`spring.application.name`).

11. Git y control de versiones

- Rama principal `main`; los commits usan conventional commits: `feat:`, `fix:`, `docs:`, `chore:`, seguidos de un resumen claro en español.
- Mensajes de commit descriptivos con present + alcance (p. ej. "fix: aplicar medidas de seguridad - eliminar credenciales expuestas").
- Los secretos, binarios generados y evidencias (GA5/, *.zip, *.eml, *.userlibraries) nunca se añaden; `.gitignore` los excluye.
- Mantener el árbol de trabajo limpio y commits atómicos por cambio.

12. DevOps: Docker y CI/CD

- Dockerfile backend multi-etapa (`maven:3.8.5-openjdk-17-slim build` → `eclipse-temurin:17-jre-alpine`) y arranque con perfil prod.
- `docker-compose.yml` define db (`postgres:16-alpine` con healthcheck), backend y frontend; volúmenes `postgres_data` y `qr_data`.
- CI (GitHub Actions): en push/PR a main ejecuta `mvn -B clean test` (backend) y `npm ci + npm run build` (frontend).
- CD: construye y publica imágenes en GHCR (`control-asistencia-backend` / `control-asistencia-frontend`) con tags `latest` y `sha`.

13. Deuda técnica y acciones recomendadas

#	Acción	Prioridad
1	Migrar <i>EmpleadoController</i> , <i>AuthController</i> y <i>AsistenciaController</i> de <code>@Autowired</code> por campo a inyección por constructor.	Alta
2	Mover la ruta de QR (<code>static/qr/</code>) a una propiedad configurable <code>app.qr.path</code> .	Alta
3	Añadir tests de integración con H2 y tests de <i>AuthController</i> , <i>AsistenciaController</i> y <i>UserController</i> .	Media
4	Estandarizar <i>Empleado</i> con Lombok (<code>@Data</code> + <code>@Builder</code> + <code>@NoArgsConstructor</code> + <code>@AllArgsConstructor</code>).	Media
5	Mejorar la respuesta JSON para tokens JWT inválidos/expirados en <i>JwtAuthenticationFilter</i> .	Media
6	Evaluar Redis para rate limiting distribuido en despliegues multi-instancia.	Baja

Criterios de aceptación para nuevas contribuciones

1. Seguir la estructura de paquetes y las convenciones de nomenclatura de la sección 4.
2. Documentar con Javadoc los métodos públicos en español.
3. Incluir tests con la convención de nombres de la sección 9.
4. No committear secretos, binarios ni artefactos de build.
5. Respetar la autorización por roles y el manejo de errores global.

